```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,ConfusionMatrixDisplay
df=pd.read_csv('/content/student_productivity_dataset_without_attendance_assignments.csv')
df
```

	student_id	age	gender	study_hours_per_day	sleep_hours	phone_usage_hours	social_media_hours	youtube_hours	gami
0	1	23	Female	4.35	3.63	3.38	2.73	1.83	
1	2	20	Male	6.14	6.58	5.48	1.51	3.13	
2	3	29	Female	4.98	3.26	4.83	3.63	0.18	
3	4	27	Female	3.19	4.58	10.06	3.95	5.75	
4	5	24	Male	7.67	6.21	3.02	1.59	5.46	
...	...	...	...	...	...	...	...	...	
20095	19154	19	Male	9.59	5.35	5.83	0.43	2.14	
20096	1890	25	Male	3.41	4.55	1.02	5.32	5.31	
20097	1686	23	Female	1.65	4.72	5.91	3.70	5.48	
20098	14384	23	Male	3.27	7.03	8.67	7.63	1.12	
20099	11433	18	Male	9.15	5.74	2.66	7.32	5.04	

20100 rows × 16 columns

Next steps: Generate code with df | New interactive sheet

```
df.shape
```

```
(20100, 16)
```

```
df.columns
```

```
Index(['student_id', 'age', 'gender', 'study_hours_per_day', 'sleep_hours',
       'phone_usage_hours', 'social_media_hours', 'youtube_hours',
       'gaming_hours', 'breaks_per_day', 'coffee_intake_mg',
       'exercise_minutes', 'stress_level', 'focus_score', 'final_grade',
       'productivity_score'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20100 entries, 0 to 20099
Data columns (total 16 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   student_id           20100 non-null  int64  
 1   age                  20100 non-null  int64  
 2   gender               20100 non-null  object 
 3   study_hours_per_day  20088 non-null  float64
 4   sleep_hours          20088 non-null  float64
 5   phone_usage_hours    20088 non-null  float64
 6   social_media_hours   20088 non-null  float64
 7   youtube_hours        20100 non-null  float64
 8   gaming_hours         20100 non-null  float64
 9   breaks_per_day       20100 non-null  int64  
 10  coffee_intake_mg     20100 non-null  int64  
 11  exercise_minutes     20084 non-null  float64
 12  stress_level         20088 non-null  float64
 13  focus_score          20088 non-null  float64
 14  final_grade          20100 non-null  float64
 15  productivity_score   20100 non-null  float64
```

```
dtypes: float64(11), int64(4), object(1)
memory usage: 2.5+ MB
```

```
df.isna().sum()
```

	0
student_id	0
age	0
gender	0
study_hours_per_day	12
sleep_hours	12
phone_usage_hours	12
social_media_hours	12
youtube_hours	0
gaming_hours	0
breaks_per_day	0
coffee_intake_mg	0
exercise_minutes	16
stress_level	12
focus_score	12
final_grade	0
productivity_score	0

dtype: int64

```
df.describe()
```

	student_id	age	study_hours_per_day	sleep_hours	phone_usage_hours	social_media_hours	youtube_hours	g
count	20100.000000	20100.000000	20088.000000	20088.000000	20088.000000	20088.000000	20100.000000	2
mean	9998.191741	23.008955	5.254375	6.516415	6.248106	4.004575	2.989822	
std	5771.846588	3.752574	2.742370	2.029547	3.314044	2.305751	1.729558	
min	1.000000	17.000000	0.500000	3.000000	0.500000	0.000000	0.000000	
25%	5002.750000	20.000000	2.900000	4.770000	3.370000	2.000000	1.490000	
50%	9993.500000	23.000000	5.250000	6.500000	6.235000	4.010000	2.980000	
75%	14995.250000	26.000000	7.640000	8.302500	9.100000	5.970000	4.480000	
max	20000.000000	29.000000	10.000000	10.000000	12.000000	8.000000	6.000000	

```
df["study_hours_per_day"] = df["study_hours_per_day"].fillna(df["study_hours_per_day"].median())
df["sleep_hours"] = df["sleep_hours"].fillna(df["sleep_hours"].median())
df["phone_usage_hours"] = df["phone_usage_hours"].fillna(df["phone_usage_hours"].median())
df["social_media_hours"] = df["social_media_hours"].fillna(df["social_media_hours"].median())
df["exercise_minutes"] = df["exercise_minutes"].fillna(df["exercise_minutes"].median())
df["stress_level"] = df["stress_level"].fillna(df["stress_level"].median())
df["focus_score"] = df["focus_score"].fillna(df["focus_score"].median())
df["gender"] = df["gender"].fillna(df["gender"].mode()[0])
```

```
df.isna().sum()
```

	0
student_id	0
age	0
gender	0
study_hours_per_day	0
sleep_hours	0
phone_usage_hours	0
social_media_hours	0
youtube_hours	0
gaming_hours	0
breaks_per_day	0
coffee_intake_mg	0
exercise_minutes	0
stress_level	0
focus_score	0
final_grade	0
productivity_score	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(100)
```

```
df = df.drop_duplicates()
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df["gender"] = le.fit_transform(df["gender"])
```

```
/tmp/ipykernel_4738/3651950995.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  df["gender"] = le.fit_transform(df["gender"])
```

```
df["total_screen_time_hours"] = (
    df["phone_usage_hours"] +
    df["youtube_hours"] +
    df["gaming_hours"] +
    df["social_media_hours"]
  )
```

```
/tmp/ipykernel_4738/3666018246.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  df["total_screen_time_hours"] = (
```

```
df = df.drop(columns=[
    "phone_usage_hours",
    "youtube_hours",
    "gaming_hours",
    "social_media_hours"
])
```

```
df.corr() ['productivity_score']
```

	productivity_score
student_id	0.012579
age	-0.005423
gender	0.003703
study_hours_per_day	0.732624
sleep_hours	0.341029
breaks_per_day	-0.005836
coffee_intake_mg	0.010190
exercise_minutes	0.002831
stress_level	-0.196902
focus_score	0.411545
final_grade	0.001954
productivity_score	1.000000
total_screen_time_hours	-0.228107

dtype: float64

```
df.drop(columns=["student_id",'age','gender','breaks_per_day','coffee_intake_mg','exercise_minutes','final_grade'],inplace=T
```

```
df
```

	study_hours_per_day	sleep_hours	stress_level	focus_score	productivity_score	total_screen_time_hours
0	4.35	3.63	10.0	57.0	33.78	13.20
1	6.14	6.58	10.0	49.0	48.99	11.85
2	4.98	3.26	2.0	38.0	36.60	13.35
3	3.19	4.58	6.0	50.0	19.87	22.28
4	7.67	6.21	6.0	41.0	52.90	15.72
...	...	...	...	...	...	...
19995	4.81	7.83	3.0	89.0	60.43	22.24
19996	0.83	4.49	10.0	56.0	17.84	14.88
19997	9.36	4.33	9.0	69.0	65.56	11.69
19998	0.74	4.44	5.0	53.0	28.06	17.26
19999	3.64	6.35	9.0	65.0	30.37	15.26

20000 rows × 6 columns

Next steps: Generate code with df | New interactive sheet

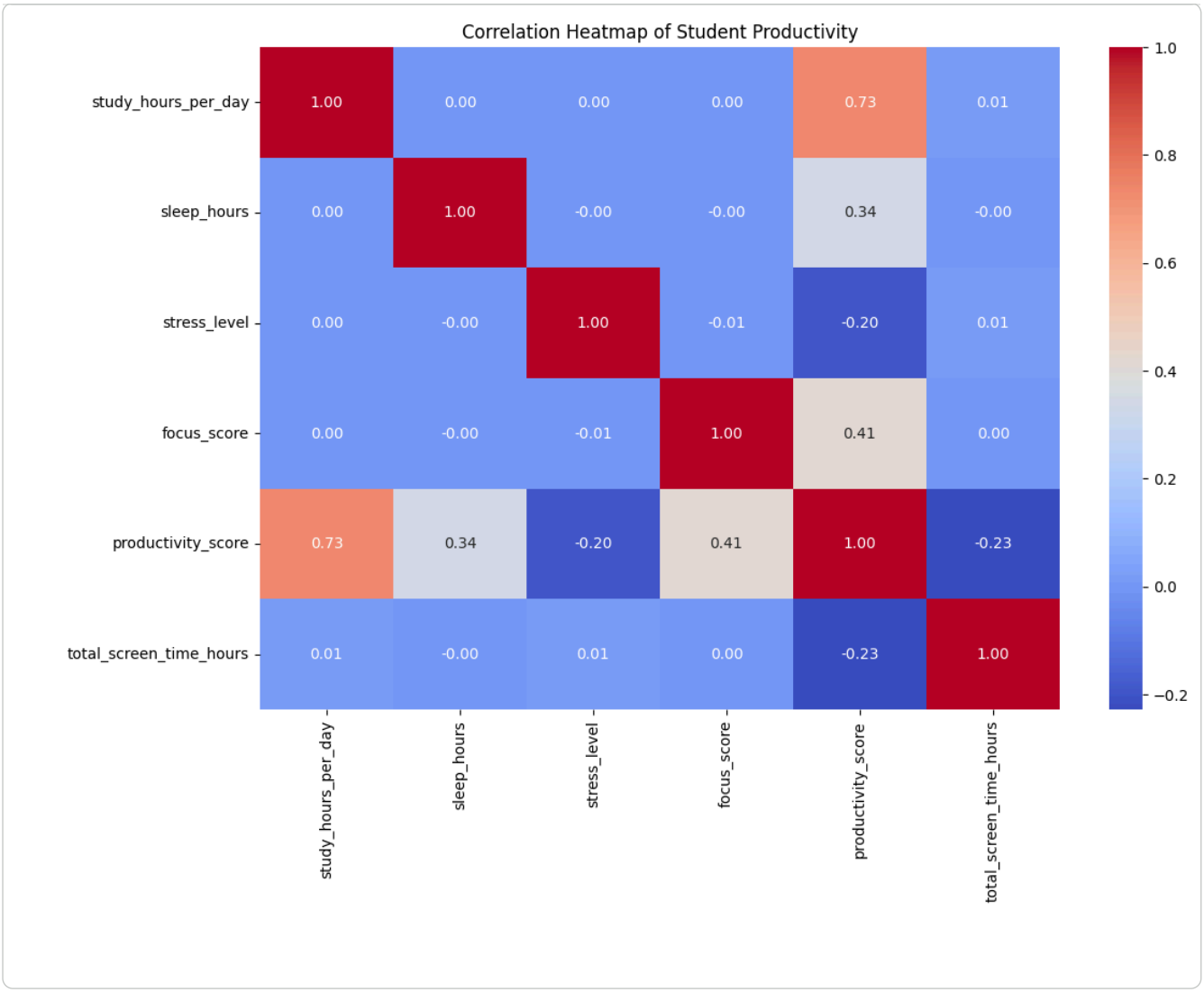
```
df["productivity_category"] = pd.cut(
    df["productivity_score"],
    bins=[-float("inf"), 40, 70, float("inf")],
    labels=["Low", "Medium", "High"]
)
```

```
plt.figure(figsize=(12,8))

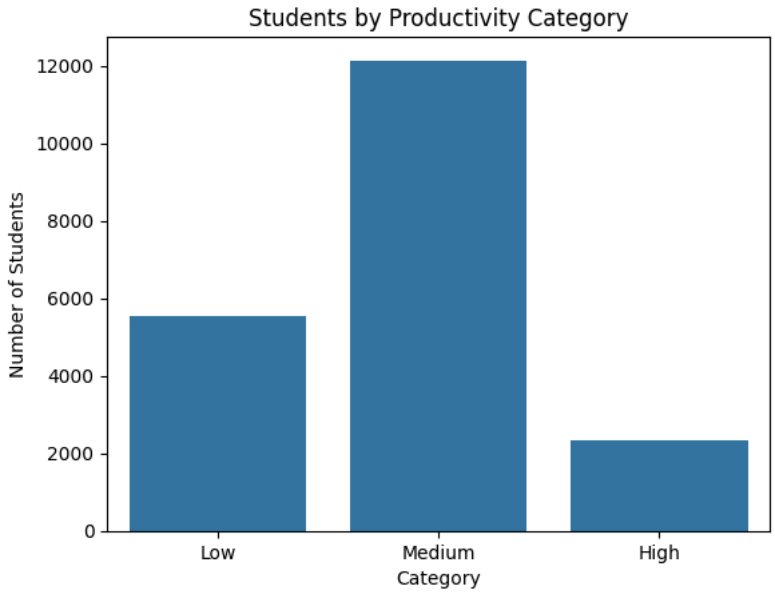
correlation = df.corr(numeric_only=True)

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap of Student Productivity")
plt.show()
```




```
sns.countplot(data=df, x="productivity_category", order=["Low", "Medium", "High"])
plt.title("Students by Productivity Category")
plt.xlabel("Category")
plt.ylabel("Number of Students")
plt.show()
```



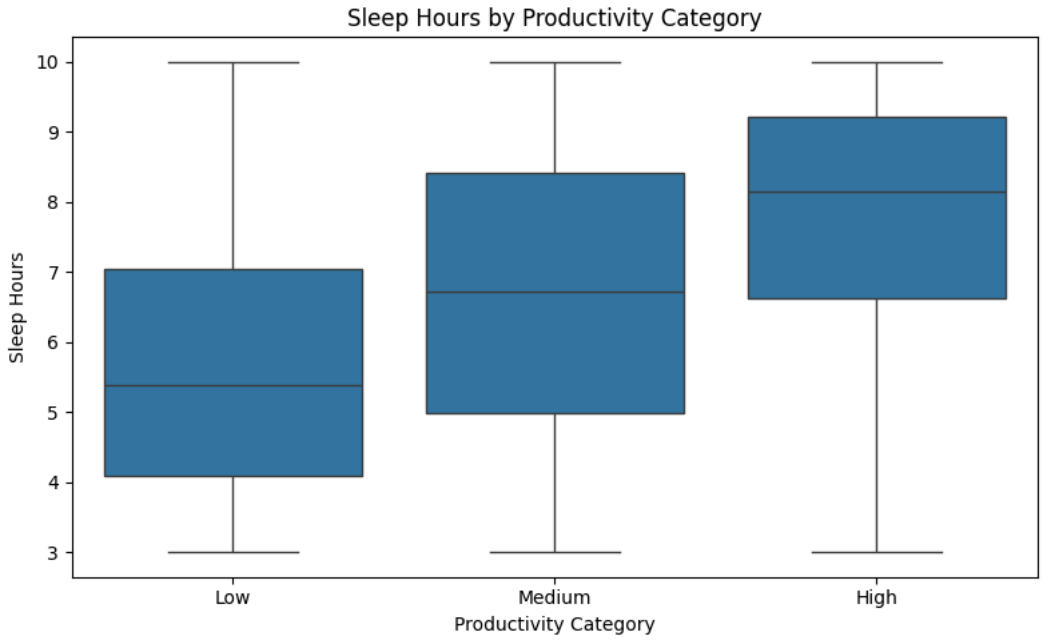

```
plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="productivity_category",
    y="sleep_hours",
```

```
)

plt.title("Sleep Hours by Productivity Category")
plt.xlabel("Productivity Category")
plt.ylabel("Sleep Hours")

plt.tight_layout()
plt.show()
```

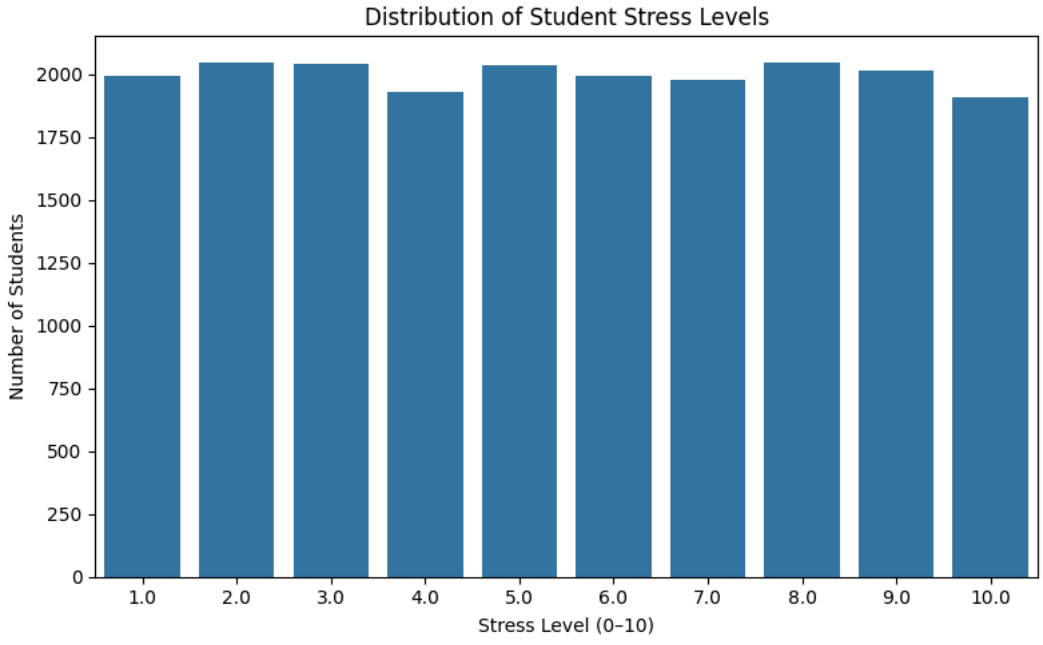



```
plt.figure(figsize=(8, 5))

sns.countplot(
    data=df,
    x="stress_level"
)

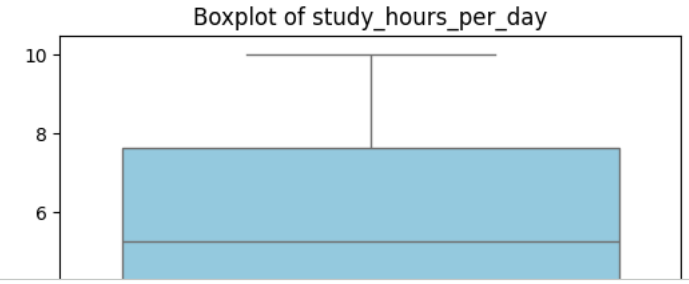
plt.title("Distribution of Student Stress Levels")
plt.xlabel("Stress Level (0–10)")
plt.ylabel("Number of Students")

plt.tight_layout()
plt.show()
```



```
# Select only numeric columns that actually exist in the current df
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
if "productivity_score" in numeric_cols:
    numeric_cols.remove("productivity_score") # Remove target score if you only want feature outliers

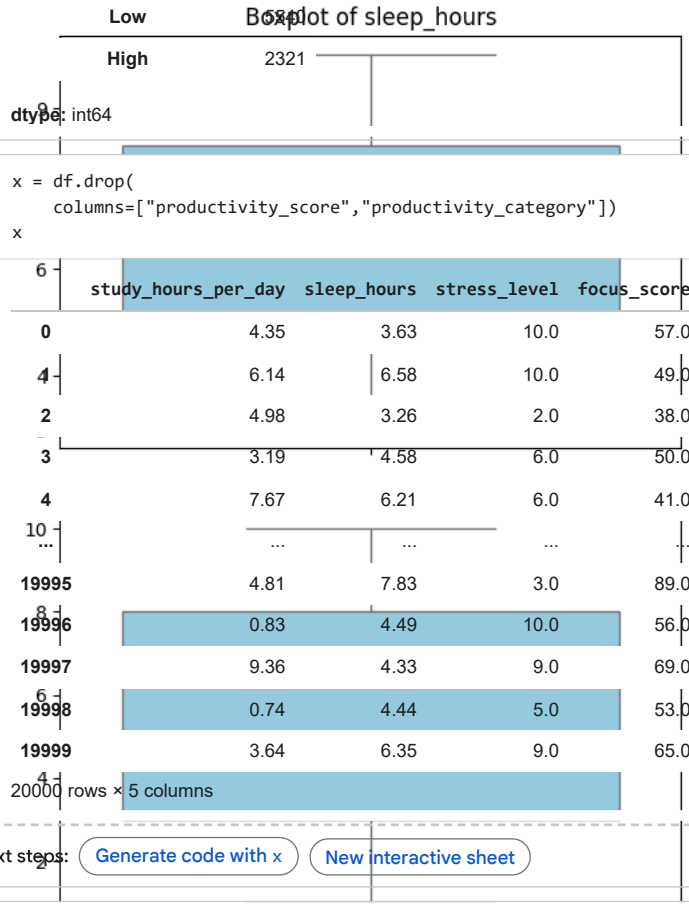
# Create boxplots for the current columns
for col in numeric_cols:
    plt.figure(figsize=(6, 4))
    sns.boxplot(data=df, y=col, color='skyblue')
    plt.title(f'Boxplot of {col}')
    plt.ylabel('')
    plt.show()
```




```
df["productivity_category"].value_counts()
```

productivity_category	count
Medium	12139
Low	5321
High	2321

dtype: int64




```
x = df.drop(
    columns=["productivity_score","productivity_category"])
x
```

	study_hours_per_day	sleep_hours	stress_level	focus_score	total_screen_time_hours
0	4.35	3.63	10.0	57.0	13.20
1	6.14	6.58	10.0	49.0	11.85
2	4.98	3.26	2.0	38.0	13.35
3	3.19	4.58	6.0	50.0	22.28
4	7.67	6.21	6.0	41.0	15.72
...	...	...	...	...	...
19995	4.81	7.83	3.0	89.0	22.24
19996	0.83	4.49	10.0	56.0	14.88
19997	9.36	4.33	9.0	69.0	11.69
19998	0.74	4.44	5.0	53.0	17.26
19999	3.64	6.35	9.0	65.0	15.26

20000 rows × 5 columns

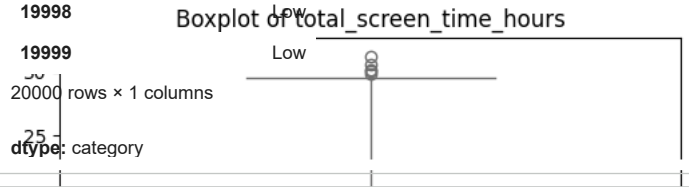
Next steps: Generate code with x | New interactive sheet

```
y = df["productivity_category"]
y
```

	productivity_category
0	Low
1	Medium
2	Low
3	Low
4	Medium
...	...
19995	Medium
19996	Low
19997	Medium
19998	Low
19999	Low

20000 rows × 1 columns

dtype: category



```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.2,random_state=42,)
```

```
scaler=StandardScaler()
x_train=scaler.fit_transform(x_train)
x_test=scaler.transform(x_test)
```

```
y.value_counts()
```

productivity_category	count
Medium	12139
Low	5540
High	2321

dtype: int64

```
smote = SMOTE(random_state=42)
x_train, y_train = smote.fit_resample(x_train, y_train)
```

```
y_train.value_counts()
```

productivity_category	count
Low	9719
Medium	9719
High	9719

dtype: int64

```
from sklearn.metrics import accuracy_score, classification_report

models = [
    RandomForestClassifier(),
    DecisionTreeClassifier(),
    KNeighborsClassifier(),
    GaussianNB()
]

for model in models:
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)

    print("\n", type(model).__name__)
    print("Accuracy:", accuracy_score(y_test, y_pred) * 100)
    print(classification_report(y_test, y_pred))
```

```
 RandomForestClassifier
Accuracy: 84.8
              precision    recall  f1-score   support

        High       0.73      0.79      0.76       484
         Low       0.81      0.87      0.84      1096
      Medium       0.90      0.85      0.87      2420

    accuracy                           0.85      4000
   macro avg       0.81      0.84      0.82      4000
weighted avg       0.85      0.85      0.85      4000


 DecisionTreeClassifier
Accuracy: 79.10000000000001
              precision    recall  f1-score   support

        High       0.65      0.71      0.68       484
         Low       0.75      0.80      0.77      1096
      Medium       0.84      0.80      0.82      2420

    accuracy                           0.79      4000
   macro avg       0.75      0.77      0.76      4000
weighted avg       0.79      0.79      0.79      4000
```

```
 KNeighborsClassifier
Accuracy: 80.22500000000001
              precision    recall  f1-score   support

        High       0.61      0.83      0.71       484
         Low       0.75      0.89      0.81      1096
      Medium       0.90      0.76      0.82      2420

    accuracy                           0.80      4000
   macro avg       0.75      0.83      0.78      4000
weighted avg       0.82      0.80      0.81      4000


 GaussianNB
Accuracy: 77.125
              precision    recall  f1-score   support

        High       0.55      0.88      0.68       484
         Low       0.71      0.91      0.80      1096
      Medium       0.91      0.69      0.78      2420

    accuracy                           0.77      4000
   macro avg       0.72      0.83      0.75      4000
weighted avg       0.81      0.77      0.77      4000
```

```
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()
```

